

Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет програмних систем та прикладної математики
Кафедра системного програмування і спеціалізованих комп’ютерних систем

ЛАБОРАТОРНА РОБОТА №»1

з дисципліни

“Розробка програмного забезпечення систем реального часу”

Тема: «Вступ до ROS 2, налаштування середовища та розробка вузлів (Nodes)»

Виконав: Ляшенко Д.Д.
Студент групи КВ-33

Київ 2026

Мета лабораторної роботи:

Опанувати засоби контейнеризації середовища розробки (Docker), навчитися створювати та збирати ROS 2 пакети за допомогою утиліти colcon, проектувати вузли у вигляді компонентів (Components) для роботи у межах єдиного процесу-виконавця (Executor) та реалізовувати вузли з керованим життєвим циклом (Lifecycle Nodes).

Порядок виконання лабораторної роботи:

1. Налаштування середовища та робочого простору:
   - 1.1 Підготувати Dockerfile на основі офіційного образу ROS 2 та створити скрипт entrypoint.sh для автоматичного сорсингу системного setup.bash.
   - 1.2 Створити структуру робочої теки ros2_ws/src, ініціалізувати C++ пакет за допомогою розширених команд colcon.
   - 1.3 Виконати збирання проєкту через colcon build –symlink-install, виконати сорсинг install/setup.bash та перевірити працездатність.
2. Розробка вузла-компонента (Component Node):
   - 2.1 Створити C++ клас вузла у власному namespace, успадкований від rclcpp::Node, який приймає конструктор rclcpp::NodeOptions. Сам конструктор можна поки лишити порожнім.
   - 2.2 Зареєструвати вузол як компонент за допомогою макросу RCLCPP_COMPONENTS_REGISTER_NODE.
   - 2.3 Налаштуйте CMakeLists.txt для збирання спільної бібліотеки з допомогою add_library(ваш_клас SHARED шлях/до_файлу.cpp), пропишіть dependencies та зареєструйте назву через rclcpp_components_register_nodes.
3. Розробка керованих вузлів (Lifecycle Nodes) та супервізора:
   - 3.1 Створити два керовані вузли (Managed Nodes), з бібліотеки LifecycleNode.
   - 3.2 Сконфігурувати і активувати ці вузли відповідними командами Transition.

Хід виконання лабораторної роботи:

Налаштування середовища та робочого простору:

dockerfile.dockerfile

```
FROM osrf/ros:humble-desktop

# Автоматичний сорсинг ROS-оточення при вході в контейнер
COPY entrypoint.sh /entrypoint.sh
RUN chmod +x /entrypoint.sh

WORKDIR /ros2_ws
ENTRYPOINT ["/entrypoint.sh"]
CMD ["bash"]
```

entrypoint.sh

```
#!/bin/bash
set -e
source /opt/ros/$ROS_DISTRO/setup.bash
if [ -f /ros2_ws/install/setup.bash ]; then
  source /ros2_ws/install/setup.bash
fi
exec "$@"
```

Windows PowerShell

```
PS C:\Users\Даня\Desktop\lections _4.1\РПЗСРЧ> docker build -t my_ros2_image .
[+] Building 575.6s (9/9) FINISHED                                                
```

```
PS C:\Users\Даня\Desktop\lections _4.1\РПЗСРЧ> docker images
REPOSITORY       TAG       IMAGE ID       CREATED         SIZE
my_ros2_image    latest    53491fb17ade   2 minutes ago   4.85GB
ubuntu           latest    d1e2e92c075e   7 months ago    117MB
redis            latest    4521b581dbdd   11 months ago   200MB
cincan/binwalk   latest    45d7159e4449   4 years ago     774MB
```

Підготувати Dockerfile на основі офіційного образу ROS 2 та створити скрипт entrypoint.sh для автоматичного сорсингу системного setup.bash.



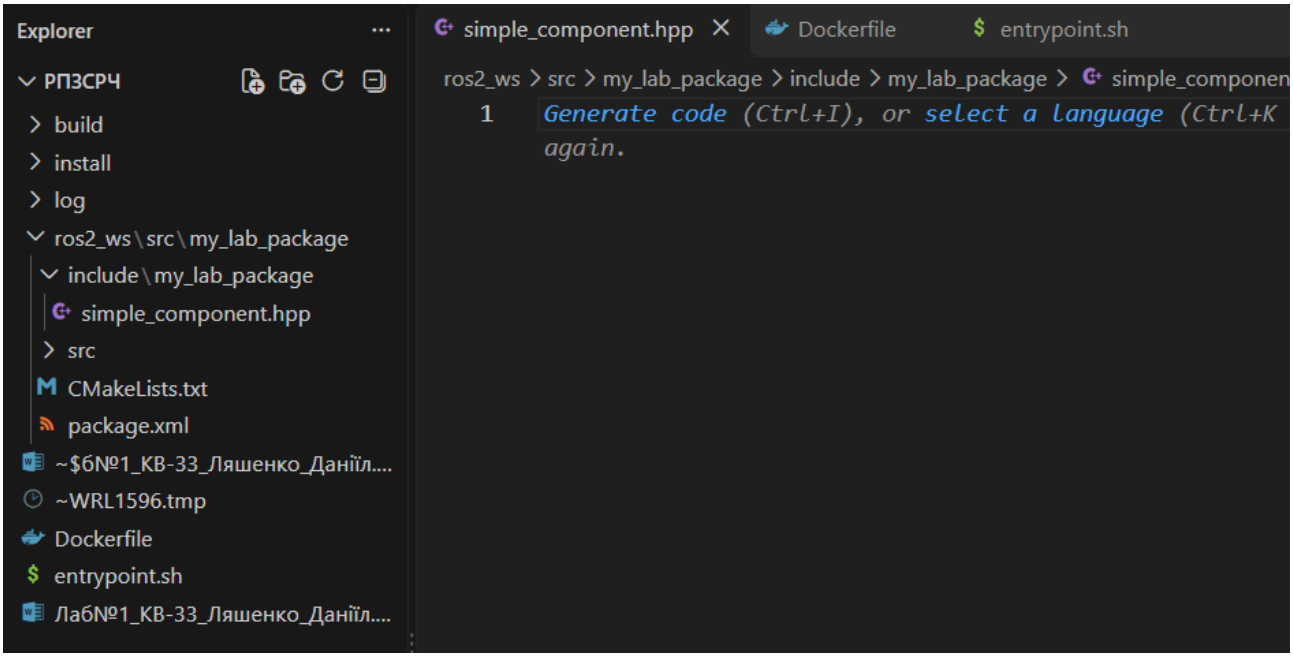
Створити структуру робочої теки ros2_ws/src, ініціалізувати C++ пакет за допомогою розширених команд colcon

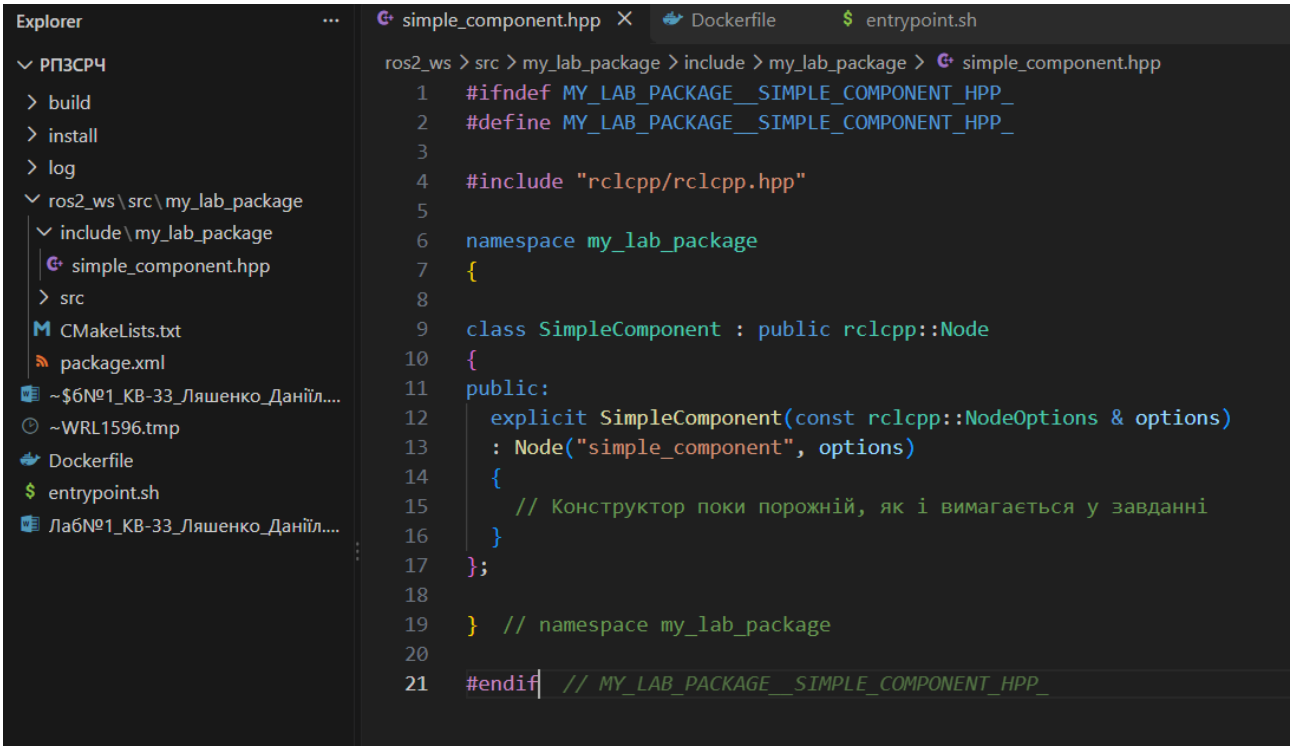
```
root@b52eb9a8e2b6:/ros2_ws/ros2_ws/src# cd /ros2_ws
root@b52eb9a8e2b6:/ros2_ws# colcon build --symlink-install
Starting >>> my_lab_package
Finished <<< my_lab_package [9.49s]

Summary: 1 package finished [10.1s]
root@b52eb9a8e2b6:/ros2_ws# source install/setup.bash
root@b52eb9a8e2b6:/ros2_ws# ros2 pkg list | grep my_lab_package
my_lab_package
```

Виконати збирання проєкту через colcon build –symlink-install, виконати сорсинг install/setup.bash та перевірити працездатність.

Розробка вузла компонента:





```
root@b52eb9a8e2b6:/ros2_ws# cd /ros2_ws/ros2_ws/src/my_lab_package
root@b52eb9a8e2b6:/ros2_ws/ros2_ws/src/my_lab_package# ls include/my_lab_package
root@b52eb9a8e2b6:/ros2_ws/ros2_ws/src/my_lab_package# nano include/my_lab_package/simple_component.hpp
bash: nano: command not found
root@b52eb9a8e2b6:/ros2_ws/ros2_ws/src/my_lab_package# cat include/my_lab_package/simple_component.hpp
#ifndef MY_LAB_PACKAGE__SIMPLE_COMPONENT_HPP_
#define MY_LAB_PACKAGE__SIMPLE_COMPONENT_HPP_

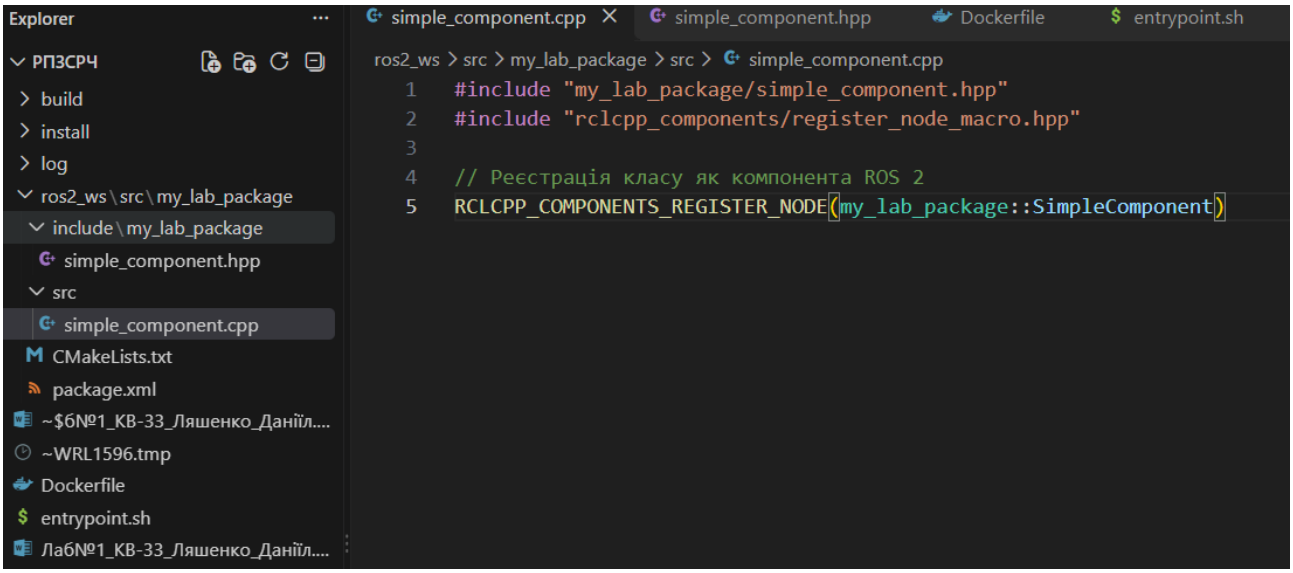
#include "rclcpp/rclcpp.hpp"

namespace my_lab_package
{

class SimpleComponent : public rclcpp::Node
{
public:
  explicit SimpleComponent(const rclcpp::NodeOptions & options)
  : Node("simple_component", options)
  {
    // Конструктор поки порожній, як і вимагається у завданні
  }
};

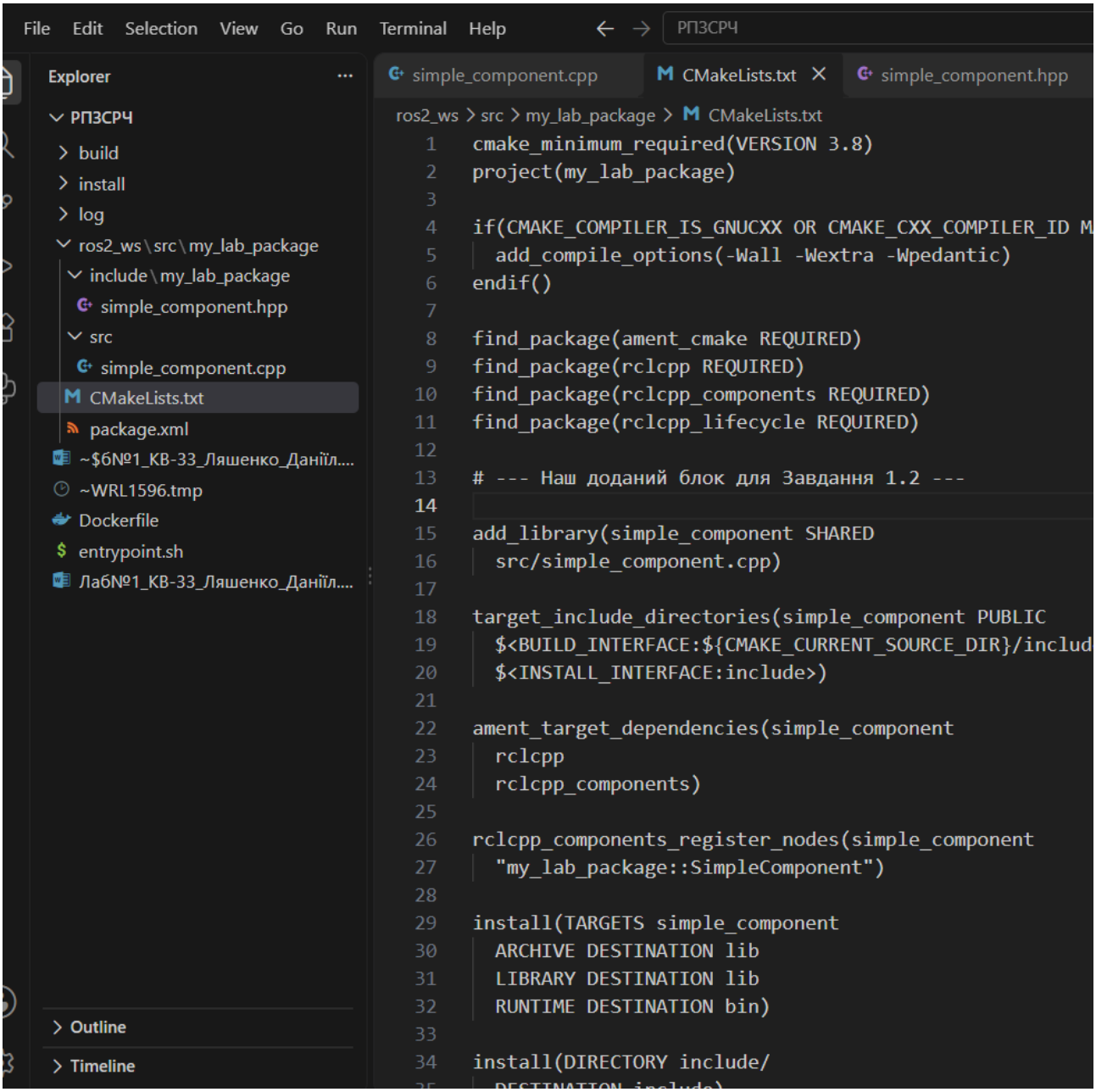
}  // namespace my_lab_package

#endifroot@b52eb9a8e2b6:/ros2_ws/ros2_ws/src/my_lab_package#
```




```
#endifroot@b52eb9a8e2b6:/ros2_ws/ros2_ws/src/my_lab_package# cat src/simple_component.cpp
#include "my_lab_package/simple_component.hpp"
#include "rclcpp_components/register_node_macro.hpp"

// Реєстрація класу як компонента ROS 2
RCLCPP_COMPONENTS_REGISTER_NODE(my_lab_package::SimpleComponent)root@b52eb9a8e2b6:/ros2_ws/ros2_ws/src/my_lab_package#
```


File Edit Selection View Go Run Terminal Help
РПЗСРЧ
Explorer
РПЗСРЧ
build
install
log
ros2_ws\src\my_lab_package
include\my_lab_package
simple_component.hpp
src
simple_component.cpp
CMakeLists.txt
package.xml
~$6№1_КВ-33_Ляшенко_Данїл....
~WRL1596.tmp
Dockerfile
entrypoint.sh
Лаб№1_КВ-33_Ляшенко_Данїл....
Outline
Timeline
simple_component.cpp
CMakeLists.txt
simple_component.hpp
ros2_ws > src > my_lab_package > CMakeLists.txt
cmake_minimum_required(VERSION 3.8)
project(my_lab_package)
if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID M
add_compile_options(-Wall -Wextra -Wpedantic)
endif()
find_package(ament_cmake REQUIRED)
find_package(rclcpp REQUIRED)
find_package(rclcpp_components REQUIRED)
find_package(rclcpp_lifecycle REQUIRED)
--- Наш доданий блок для Завдання 1.2 ---
add_library(simple_component SHARED
src/simple_component.cpp)
target_include_directories(simple_component PUBLIC
$<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/includ
$<INSTALL_INTERFACE:include>)
ament_target_dependencies(simple_component
rclcpp
rclcpp_components)
rclcpp_components_register_nodes(simple_component
"my_lab_package::SimpleComponent")
install(TARGETS simple_component
ARCHIVE DESTINATION lib
LIBRARY DESTINATION lib
RUNTIME DESTINATION bin)
install(DIRECTORY include/

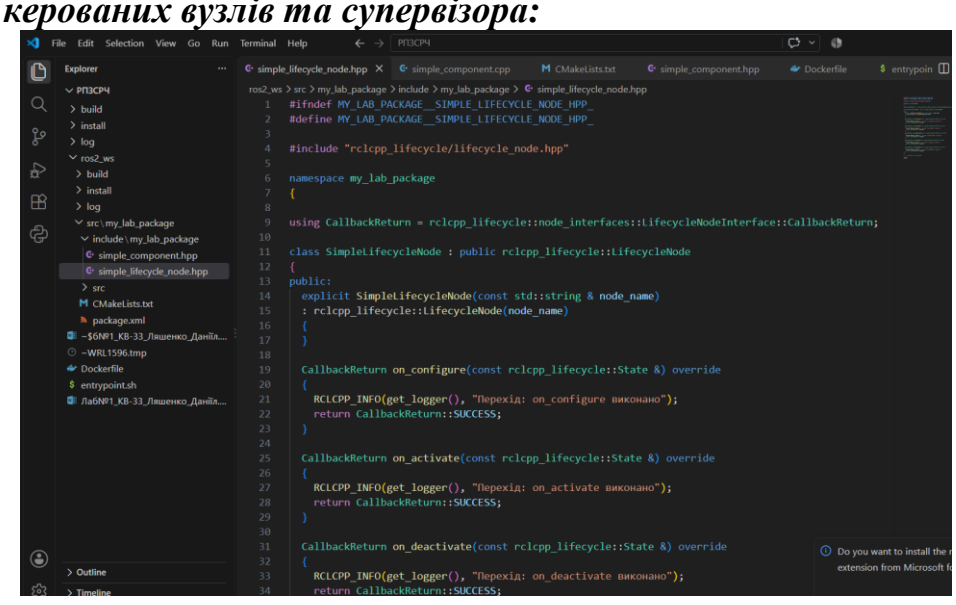
```
root@b52eb9a8e2b6:/ros2_ws/ros2_ws# colcon build --symlink-install
Starting >>> my_lab_package
Finished <<< my_lab_package [19.4s]

Summary: 1 package finished [19.7s]
root@b52eb9a8e2b6:/ros2_ws/ros2_ws# source install/setup.bash
root@b52eb9a8e2b6:/ros2_ws/ros2_ws# ros2 component types | grep simple_component
root@b52eb9a8e2b6:/ros2_ws/ros2_ws# ros2 component types | grep simple_component
root@b52eb9a8e2b6:/ros2_ws/ros2_ws# ros2 component types | grep my_lab_package
my_lab_package
  my_lab_package::SimpleComponent
root@b52eb9a8e2b6:/ros2_ws/ros2_ws# ros2 component types
my_lab_package
  my_lab_package::SimpleComponent
tf2_ros
  tf2_ros::StaticTransformBroadcasterNode
robot_state_publisher
  robot_state_publisher::RobotStatePublisher
teleop_twist_joy
  teleop_twist_joy::TeleopTwistJoy
examples_rclcpp_minimal_subscriber
  WaitSetSubscriber
  StaticWaitSetSubscriber
  TimeTriggeredWaitSetSubscriber
demo_nodes_cpp
  demo_nodes_cpp::OneOffTimerNode
  demo_nodes_cpp::ReuseTimerNode
  demo_nodes_cpp::ServerNode
  demo_nodes_cpp::ClientNode
  demo_nodes_cpp::ListParameters
```

```
root@b52eb9a8e2b6: /ros2_w
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Даня\Desktop\lections _4.1\РПЗСРЧ> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS          PORTS     NAMES
b52eb9a8e2b6   my_ros2_image   "/entrypoint.sh bash"   36 minutes ago   Up 36 minutes             trusting_thompson
PS C:\Users\Даня\Desktop\lections _4.1\РПЗСРЧ> docker exec -it b52eb9a8e2b6 bash
root@b52eb9a8e2b6:/ros2_ws# source /ros2_ws/ros2_ws/install/setup.bash
root@b52eb9a8e2b6:/ros2_ws# ros2 component load /ComponentManager my_lab_package my_lab_package::SimpleComponent
Loaded component 1 into '/ComponentManager' container node as '/simple_component'
root@b52eb9a8e2b6:/ros2_ws# ros2 component list
/ComponentManager
  1  /simple_component
root@b52eb9a8e2b6:/ros2_ws#
```

Розробка керованих вузлів та супервізора:



```
PS C:\Users\Даня\Desktop\lections _4.1\РПЗСРЧ> docker exec -it b52eb9a8e2b6 bash
root@b52eb9a8e2b6:/ros2_ws# cd /ros2_ws/ros2_ws/src/my_lab_package
root@b52eb9a8e2b6:/ros2_ws/ros2_ws/src/my_lab_package# cat include/my_lab_package/simple_lifecycle_node.hpp
#ifndef MY_LAB_PACKAGE__SIMPLE_LIFECYCLE_NODE_HPP_
#define MY_LAB_PACKAGE__SIMPLE_LIFECYCLE_NODE_HPP_

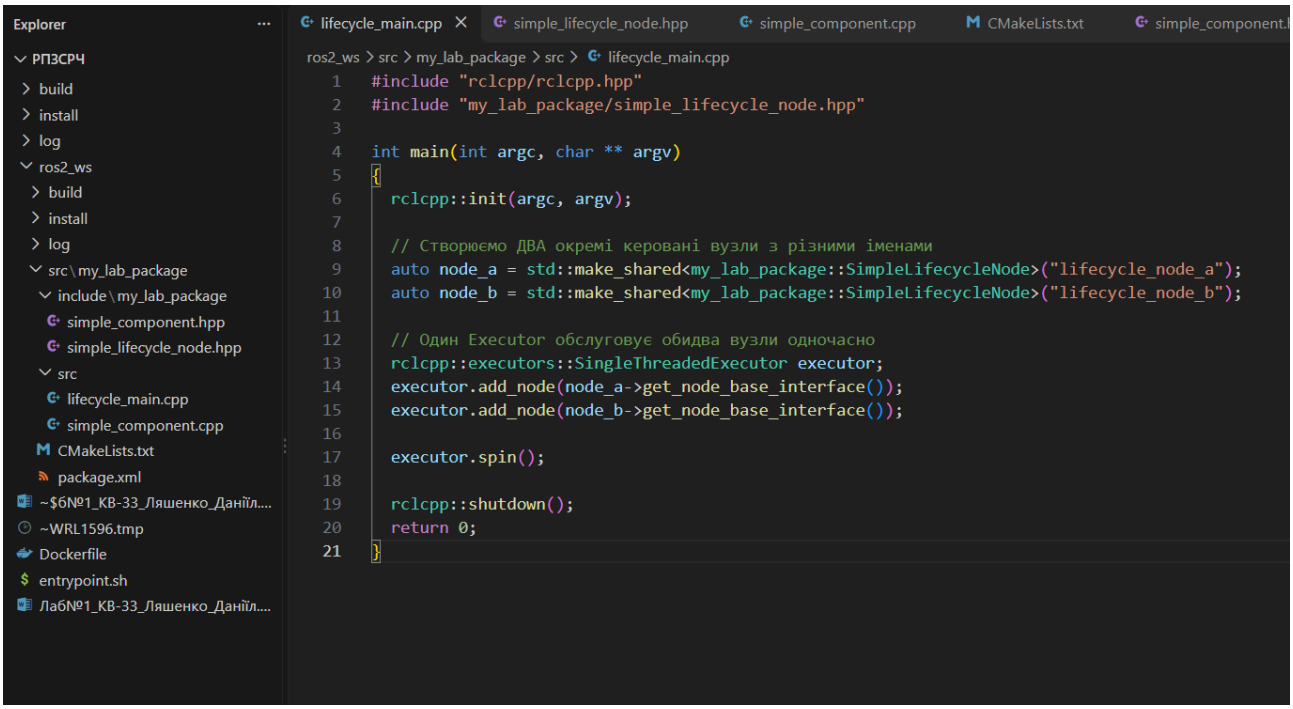
#include "rclcpp_lifecycle/lifecycle_node.hpp"

namespace my_lab_package
{

using CallbackReturn = rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn;

class SimpleLifecycleNode : public rclcpp_lifecycle::LifecycleNode
{
public:
  explicit SimpleLifecycleNode(const std::string & node_name)
  : rclcpp_lifecycle::LifecycleNode(node_name)
  {
  }

  CallbackReturn on_configure(const rclcpp_lifecycle::State &) override
  {
    RCLCPP_INFO(get_logger(), "Перехід: on_configure виконано");
    return CallbackReturn::SUCCESS;
```




```
root@b52eb9a8e2b6:/ros2_ws/ros2_ws/src/my_lab_package# cat src/lifecycle_main.cpp
#include "rclcpp/rclcpp.hpp"
#include "my_lab_package/simple_lifecycle_node.hpp"

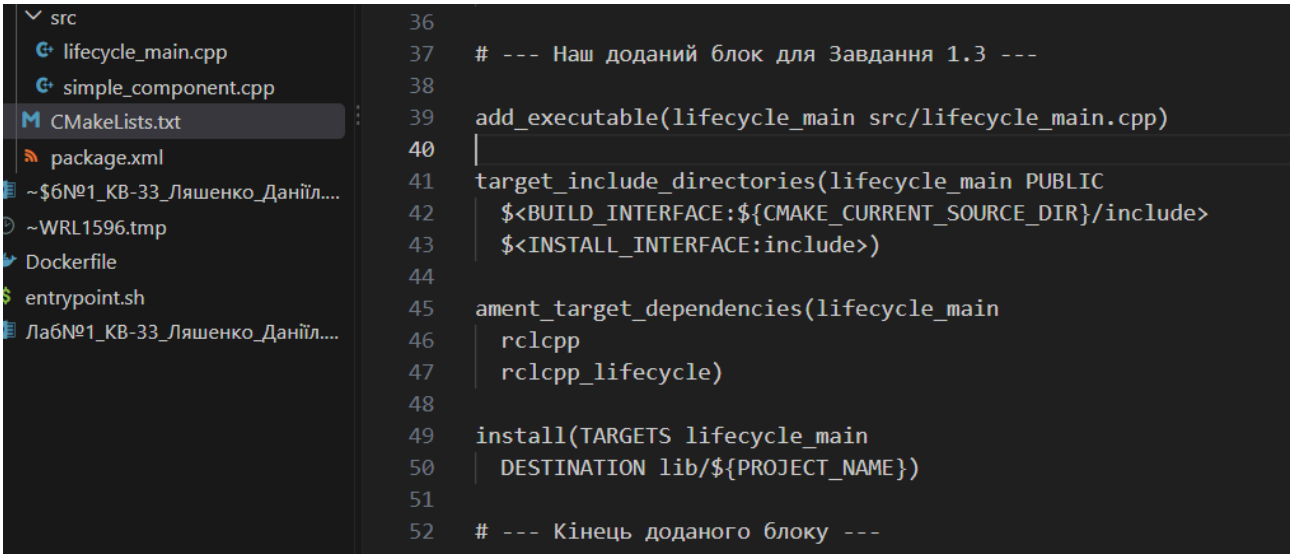
int main(int argc, char ** argv)
{
  rclcpp::init(argc, argv);

  // Створюємо ДВА окремі керовані вузли з різними іменами
  auto node_a = std::make_shared<my_lab_package::SimpleLifecycleNode>("lifecycle_node_a");
  auto node_b = std::make_shared<my_lab_package::SimpleLifecycleNode>("lifecycle_node_b");

  // Один Executor обслуговує обидва вузли одночасно
  rclcpp::executors::SingleThreadedExecutor executor;
  executor.add_node(node_a->get_node_base_interface());
  executor.add_node(node_b->get_node_base_interface());

  executor.spin();

  rclcpp::shutdown();
  return 0;
}root@b52eb9a8e2b6:/ros2_ws/ros2_ws/src/my_lab_package#
```




```
  ARCHIVE DESTINATION lib
  LIBRARY DESTINATION lib
  RUNTIME DESTINATION bin)

install(DIRECTORY include/
  DESTINATION include)

# --- Наш доданий блок для Завдання 1.3 ---

add_executable(lifecycle_main src/lifecycle_main.cpp)

target_include_directories(lifecycle_main PUBLIC
  $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
  $<INSTALL_INTERFACE:include>)

ament_target_dependencies(lifecycle_main
  rclcpp
  rclcpp_lifecycle)

install(TARGETS lifecycle_main
  DESTINATION lib/${PROJECT_NAME})

# --- Кінець доданого блоку ---

# --- Кінець доданого блоку ---

if(BUILD_TESTING)
  find_package(ament_lint_auto REQUIRED)
  ament_lint_auto_find_test_dependencies()
endif()

ament_package()root@b52eb9a8e2b6:/ros2_ws/ros2_ws/src/my_lab_package#
```

```
root@b52eb9a8e2b6: /ros2_w
Starting >>> my_lab_package
--- stderr: my_lab_package
Traceback (most recent call last):
  File "/opt/ros/humble/share/ament_cmake_core/cmake/package_templates/templates_2_cmake.py", line 21, in <module>
    from ament_package.templates import get_environment_hook_template_path
ModuleNotFoundError: No module named 'ament_package'
CMake Error at /opt/ros/humble/share/ament_cmake_core/cmake/ament_cmake_package_templates-extras.cmake:41 (message):
  execute_process(/usr/bin/python3
  /opt/ros/humble/share/ament_cmake_core/cmake/package_templates/templates_2_cmake.py
  /ros2_ws/ros2_ws/build/my_lab_package/ament_cmake_package_templates/templates.cmake)
  returned error code 1
Call Stack (most recent call first):
  /opt/ros/humble/share/ament_cmake_core/cmake/ament_cmake_coreConfig.cmake:41 (include)
  /opt/ros/humble/share/ament_cmake/cmake/ament_cmake_export_dependencies-extras.cmake:15 (find_package)
  /opt/ros/humble/share/ament_cmake/cmake/ament_cmakeConfig.cmake:41 (include)
  CMakeLists.txt:8 (find_package)


gmake: *** [Makefile:267: cmake_check_build_system] Error 1
---
Failed   <<< my_lab_package [1.12s, exited with code 2]

Summary: 0 packages finished [1.54s]
  1 package failed: my_lab_package
  1 package had stderr output: my_lab_package
root@b52eb9a8e2b6:/ros2_ws/ros2_ws# python3 -c "import ament_package; print(ament_package.__file__)"
Traceback (most recent call last):
  File "<string>", line 1, in <module>
ModuleNotFoundError: No module named 'ament_package'
root@b52eb9a8e2b6:/ros2_ws/ros2_ws# source /opt/ros/humble/setup.bash
root@b52eb9a8e2b6:/ros2_ws/ros2_ws# python3 -c "import ament_package; print(ament_package.__file__)"
/opt/ros/humble/lib/python3.10/site-packages/ament_package/__init__.py
root@b52eb9a8e2b6:/ros2_ws/ros2_ws# cd /ros2_ws/ros2_ws
root@b52eb9a8e2b6:/ros2_ws/ros2_ws# colcon build --symlink-install
Starting >>> my_lab_package
Finished <<< my_lab_package [11.8s]

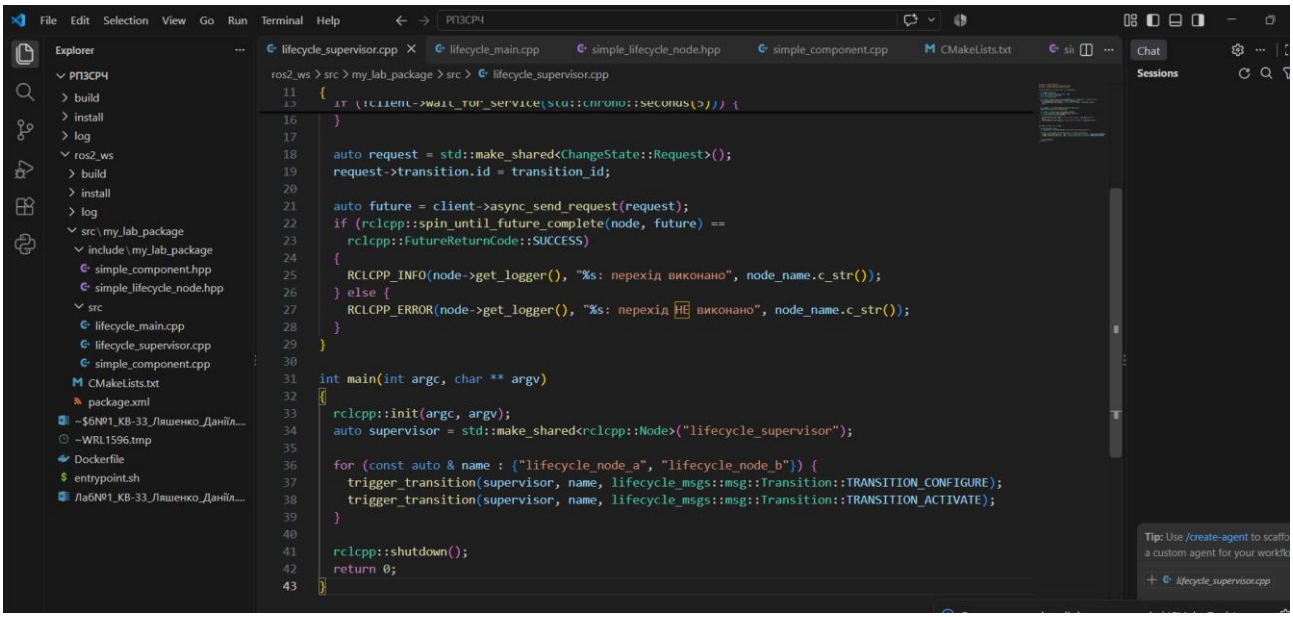
Summary: 1 package finished [12.2s]
root@b52eb9a8e2b6:/ros2_ws/ros2_ws# source install/setup.bash
root@b52eb9a8e2b6:/ros2_ws/ros2_ws# ros2 run my_lab_package lifecycle_main
```

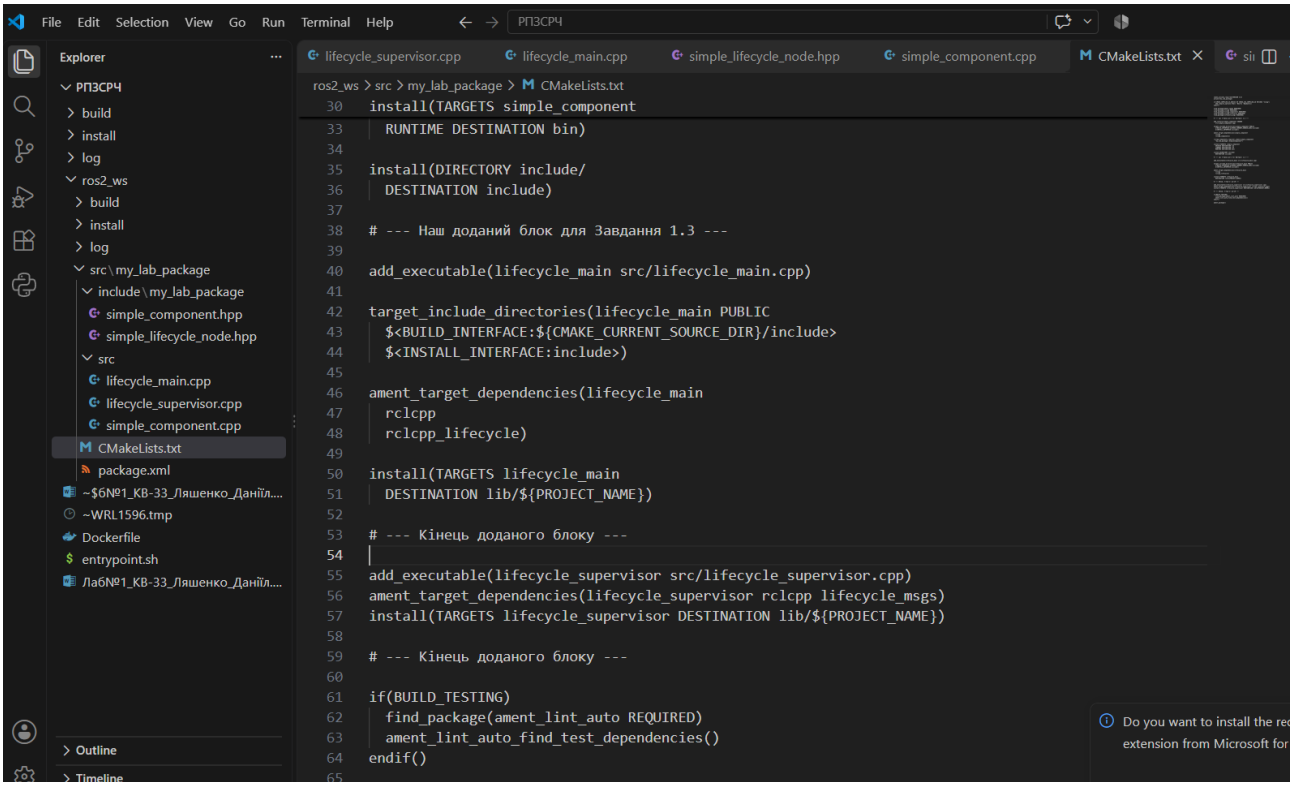
```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Даня\Desktop\lections _4.1\РПЗСРЧ>  docker exec -it b52eb9a8e2b6 bash
root@b52eb9a8e2b6:/ros2_ws# source /opt/ros/humble/setup.bash
root@b52eb9a8e2b6:/ros2_ws# source /ros2_ws/ros2_ws/install/setup.bash
root@b52eb9a8e2b6:/ros2_ws# ros node list
bash: ros: command not found
root@b52eb9a8e2b6:/ros2_ws# docker ps
bash: docker: command not found
root@b52eb9a8e2b6:/ros2_ws# exec
root@b52eb9a8e2b6:/ros2_ws# exit
exit
PS C:\Users\Даня\Desktop\lections _4.1\РПЗСРЧ> docker ps
CONTAINER ID   IMAGE           COMMAND                  CREATED          STATUS          PORTS     NAMES
b52eb9a8e2b6   my_ros2_image   "/entrypoint.sh bash"   57 minutes ago   Up 57 minutes             trusting_thompson
PS C:\Users\Даня\Desktop\lections _4.1\РПЗСРЧ> docker exec -it b52eb9a8e2b6 bash
root@b52eb9a8e2b6:/ros2_ws# source /opt/ros/humble/setup.bash
root@b52eb9a8e2b6:/ros2_ws# source /ros2_ws/ros2_ws/install/setup.bash
root@b52eb9a8e2b6:/ros2_ws# ros node list
bash: ros: command not found
root@b52eb9a8e2b6:/ros2_ws# ros2 node list
/ComponentManager
/lifecycle_node_a
/lifecycle_node_b
/simple_component
```

```
root@b52eb9a8e2b6:/ros2_ws# ros2 lifecycle get /lifecycle_node_a
unconfigured [1]
root@b52eb9a8e2b6:/ros2_ws# ros2 lifecycle set /lifecycle_node_a configure
Transitioning successful
root@b52eb9a8e2b6:/ros2_ws# ros2 lifecycle set /lifecycle_node_a activate
Transitioning successful
root@b52eb9a8e2b6:/ros2_ws# ros2 lifecycle set /lifecycle_node_b configure
Transitioning successful
root@b52eb9a8e2b6:/ros2_ws# ros2 lifecycle set /lifecycle_node_b activate
Transitioning successful
root@b52eb9a8e2b6:/ros2_ws# ros2 lifecycle get /lifecycle_node_a
active [3]
root@b52eb9a8e2b6:/ros2_ws# ros2 lifecycle get /lifecycle_node_b
active [3]
root@b52eb9a8e2b6:/ros2_ws#
```

```
root@b52eb9a8e2b6:/ros2_ws/ros2_ws# ros2 run my_lab_package lifecycle_main
[INFO] [1790507402.102461478] [lifecycle_node_a]: Перехід: on_configure виконано
[INFO] [1790507410.823004009] [lifecycle_node_a]: Перехід: on_activate виконано
[INFO] [1790507421.623042071] [lifecycle_node_b]: Перехід: on_configure виконано
[INFO] [1790507433.073167460] [lifecycle_node_b]: Перехід: on_activate виконано
```






```
root@b52eb9a8e2b6:/ros2_ws# colcon build
Starting >>> my_lab_package
Finished <<< my_lab_package [16.8s]

Summary: 1 package finished [17.4s]
root@b52eb9a8e2b6:/ros2_ws#
```

```
root@b52eb9a8e2b6:/ros2_ws# source install/setup.bash
root@b52eb9a8e2b6:/ros2_ws# ros2 run my_lab_package lifecycle_supervisor
[INFO] [1790507860.915317158] [lifecycle_supervisor]: lifecycle_node_a: перехід виконано
[INFO] [1790507860.918450881] [lifecycle_supervisor]: lifecycle_node_a: перехід виконано
[INFO] [1790507860.920214468] [lifecycle_supervisor]: lifecycle_node_b: перехід виконано
[INFO] [1790507860.922296096] [lifecycle_supervisor]: lifecycle_node_b: перехід виконано
```

```
root@b52eb9a8e2b6:/ros2_ws/ros2_ws# ros2 run my_lab_package lifecycle_main
[INFO] [1790507402.102461478] [lifecycle_node_a]: Перехід: on_configure виконано
[INFO] [1790507410.823004009] [lifecycle_node_a]: Перехід: on_activate виконано
[INFO] [1790507421.623042071] [lifecycle_node_b]: Перехід: on_configure виконано
[INFO] [1790507433.073167460] [lifecycle_node_b]: Перехід: on_activate виконано
[WARN] [1790507860.914760430] [rcl_lifecycle]: No transition matching 1 found for current state active
[WARN] [1790507860.914909649] [rcl_lifecycle]: No transition matching 1 found for current state active
[ERROR] [1790507860.914983362] [lifecycle_node_a]: Unable to start transition 1 from current state active: Transition is not registered., at ./src/rcl_lifecycle.c:355
[WARN] [1790507860.918133586] [rcl_lifecycle]: No transition matching 3 found for current state active
[WARN] [1790507860.918182130] [rcl_lifecycle]: No transition matching 3 found for current state active
[ERROR] [1790507860.918214794] [lifecycle_node_a]: Unable to start transition 3 from current state active: Transition is not registered., at ./src/rcl_lifecycle.c:355
[WARN] [1790507860.919800296] [rcl_lifecycle]: No transition matching 1 found for current state active
[WARN] [1790507860.919837989] [rcl_lifecycle]: No transition matching 1 found for current state active
[ERROR] [1790507860.919875632] [lifecycle_node_b]: Unable to start transition 1 from current state active: Transition is not registered., at ./src/rcl_lifecycle.c:355
[WARN] [1790507860.921944154] [rcl_lifecycle]: No transition matching 3 found for current state active
[WARN] [1790507860.921992448] [rcl_lifecycle]: No transition matching 3 found for current state active
[ERROR] [1790507860.922028758] [lifecycle_node_b]: Unable to start transition 3 from current state active: Transition is not registered., at ./src/rcl_lifecycle.c:355
```

Висновок до лабораторної роботи:

У ході виконання лабораторної роботи було опановано базові засоби розробки робототехнічних систем на основі ROS 2. Для забезпечення відтворюваності середовища розробки було створено Docker-образ на основі офіційного дистрибутива ROS 2 з автоматичним сорсингом системного оточення

через entrypoint-скрипт, а також сформовано робочий простір (workspace) і зібрано власний C++ пакет за допомогою утиліти colcon. У межах практичної частини розроблено вузол-компонент (Component Node), реалізований як окремий клас, успадкований від rclcpp::Node, скомпільований у спільну бібліотеку (SHARED) та зареєстрований за допомогою макросу RCLCPP_COMPONENTS_REGISTER_NODE, що дозволило динамічно завантажити його в єдиний процес-виконавець (Executor). Також створено два незалежні керовані вузли (Lifecycle Nodes), для яких реалізовано та перевірено переходи між станами скінченного автомата життєвого циклу (configure, activate, deactivate, cleanup) — як вручну через CLI-команди ros2 lifecycle, так і програмно за допомогою окремого вузла-супервізора, що звертається до сервісів зміни стану. Виконана робота продемонструвала практичні переваги компонентного підходу (зниження затримок комунікації за рахунок спільної пам'яті) та важливість керованого життєвого циклу для вузлів зі складною ініціалізацією, а також дозволила закріпити навички роботи з інструментами collcon, Docker та базовим API rclcpp/rclcpp_components/rclcpp_lifecycle.